Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An Autonomous Institution Affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	VI
Subject Code & Name	UCS2612 – Machine Learning Algorithms Laboratory		
Academic Year	2025–2026 (Even)	Batch	2023–2027
Due Date	27-01-2026		

Experiment 1: Working with Python packages-Numpy, Scipy, Scikit-Learn, Matplotlib

1 Aim

To explore the various functions and methods available in the following Python libraries: Numpy, Pandas, Scipy, Scikit-learn, Matplotlib. Understand the key operations such as array manipulations, data preprocessing, mathematical computing, machine learning workflows, and data visualization. To identify the type of ML task associated with each dataset and to explore the steps involved in the machine learning workflow

2 Python Code

The following code snippets illustrate the core implementation of the classifiers:

Libraries Required

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import scipy

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report, precision_score, recall_score, f1_score
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression

import xgboost as xgb
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow import keras
```

```
import torch
import torch.nn as nn
import torch.optim as optim
import os
import time
import cv2

plt.rcParams.update({
    "font.family": "Arial",
    "font.size": 15,
    "axes.titlesize": 15,
    "axes.titleweight": "bold",
    "axes.labelsize": 15,
    "axes.labelweight": "bold",
    "xtick.labelsize": 13,
    "ytick.labelsize": 13
})
```

Dataset 1 - Loan amount prediction

```
# Loading the dataset
df=pd.read_csv("loan_data.csv")
print(df.head())

# First five rows of the dataset
df.head()

# Dataset Columns details
df.info()

# Statistics summary of the numerical columns
df.describe()

# List of columns in the dataset
df.columns

# Number of rows and columns of the Dataset
df.shape

# Checking if any null / missing values present
df.isna().sum()

# Unique values of column 'person_age'
df['person_age'].unique()

# storing numerical columns which have unique values more than 10
num_cols = [col for col in df.select_dtypes(include="number").columns if df[col].
    nunique() > 10 ]

# Plotting boxplot for numerical columns
ncols = 2
nrows = (len(num_cols) + ncols - 1) // ncols
fig, axes = plt.subplots( nrows=nrows, ncols=ncols, figsize=(20, 6 * nrows))
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.boxplot(x=df[col], ax=axes[i])
```

```
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
plt.tight_layout()
plt.show()

# Plotting histogram with kde for numerical columns
ncols = 2
nrows = (len(num_cols) + ncols - 1) // ncols
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(20, 6 * nrows))
# Flatten axes to always use single index
if isinstance(axes, np.ndarray):
    axes = axes.flatten()
else:
    axes = [axes]
for i, col in enumerate(num_cols):
    sns.histplot(df[col], kde=True, ax=axes[i])
    axes[i].set_title(f"{col} - Histogram")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
plt.tight_layout()
plt.show()

# Storing Categorical columns
cat_cols = ['person_gender', 'person_education', 'person_home_ownership', '
    loan_intent', 'previous_loan_defaults_on_file']

# Plotting bar plot for categorical columns
ncols = 3
nrows = (len(cat_cols) + ncols - 1) // ncols
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(20, 4 * nrows))
# Make axes always a flat array
if isinstance(axes, np.ndarray):
    axes = axes.flatten()
else:
    axes = [axes]
# Plot each categorical column
for i, col in enumerate(cat_cols):
    df[col].value_counts().plot(kind='bar', ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel('')
    axes[i].set_ylabel('Count')
plt.tight_layout()
plt.show()

# Plotting boxplot with columns vs loan status
ncols = 4
rows = math.ceil(len(num_cols) / ncols)
fig, axes = plt.subplots(nrows=rows, ncols=ncols,figsize=(20, 4 * rows))
axes = axes.flatten()  # easy indexing
for i, col in enumerate(num_cols):
    sns.boxplot(x='loan_status', y=col, data=df, ax=axes[i])
    axes[i].set_title(f"{col} vs Loan Status")
    axes[i].set_xlabel('loan_status', )
    axes[i].set_ylabel(col)
plt.tight_layout()
plt.show()

# Heatmap plot for correlation among features.
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(df[num_cols].corr(), annot=True)
plt.title("Correlation Matrix")
plt.show()

# Loan status distribution
plt.figure(figsize=(6,4))
sns.countplot(x="loan_status",data=df)
plt.title("Loan status distribution")
plt.xlabel('loan_status')
plt.ylabel('count')
plt.show()

# Count plot for previous loan defaults on file vs loan approval status.
plt.figure(figsize=(6, 4))
sns.countplot(x='previous_loan_defaults_on_file', hue='loan_status',data=df)
plt.title("Loan Approval vs Previous Loan Defaults")
plt.xlabel("Previous Loan Default on File")
plt.ylabel("Number of Loans")
plt.show()

# histogram plot for all columns.
cols = df.columns
n_cols = 4
n_rows = (len(cols) + n_cols - 1) // n_cols  # ceiling division
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 4 * n_rows))
axes = axes.flatten()  # make indexing easy
for i, col in enumerate(cols):
    sns.histplot(data=df, x=col, hue='loan_status', bins=30, kde=True, ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('count')
plt.tight_layout()
plt.show()

# Separating and storing numerical and categorical column features.
X_num = df.select_dtypes(include=['int64','float64'])  # numerical
X_cat = df.select_dtypes(include=['object','category'])  # categorical
y = df['loan_status']

# Scaling and Fit transform
X_num_scaled = StandardScaler().fit_transform(X_num)

# One Hot Encoder for categorical features.
X_cat_encoded = OneHotEncoder(handle_unknown="ignore").fit_transform(X_cat).
    toarray()
X_processed = np.hstack([X_num_scaled, X_cat_encoded])

#  T r a i n Test  Split
X_train, X_test, y_train, y_test = train_test_split( X_processed, y, test_size
    =0.2, random_state=42, stratify=y)

# Model Training
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Performance Metric
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
```

```
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

Dataset 2 - Predicting Diabetes

```
# Loading the dataset
df1 = pd.read_csv('diabetes.csv')
print(df1.head())

# First five rows of the dataset
df1.head(5)

# Dataset Columns details
df1.info()

# Statistics summary of the numerical columns
df1.describe()

# List of columns in the dataset
df1.columns

# Number of rows and columns of the Dataset
df1.shape

# Checking if any null / missing values present
df1.isna().sum()

# Histogram plot for numerical features.
num_cols = ['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']
ncols=2
nrows=math.ceil(len(num_cols)/ncols)
fig,axes=plt.subplots(nrows=nrows,ncols=ncols,figsize=(20,6*nrows))
axes=axes.flatten()
for i,col in enumerate(num_cols):
    sns.histplot(df1[col],kde=True,ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
for j in range(i+1,len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

# Boxplot for numerical features
ncols=2
nrows=math.ceil(len(num_cols)/ncols)
fig,axes=plt.subplots(nrows=nrows,ncols=ncols,figsize=(20,4*nrows))
axes=axes.flatten()
for i,col in enumerate(num_cols):
    sns.boxplot(df1[col],ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel('')
    axes[i].set_ylabel(col)
for j in range(i+1,len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()
```

```python
# Count plot for categorical features.
cat_cols = ['hypertension', 'heart_disease', 'diabetes']
fig, axes = plt.subplots(nrows=1, ncols=5, figsize=(20,4))
axes = axes.flatten()
for i, col in enumerate(cat_cols):
    sns.countplot(x=df1[col], ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
for j in range(i+1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

# Heatmap plot
plt.figure(figsize=(10, 6))
sns.heatmap(df1[num_cols].corr(), annot=True)
plt.title("Correlation Matrix")
plt.show()

# Histogram plot
cols = df1.columns
n_cols = 4
n_rows = (len(cols) + n_cols - 1) // n_cols  # ceiling division
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 4 * n_rows))
axes = axes.flatten()  # make indexing easy
for i, col in enumerate(cols):
    sns.histplot(data=df1, x=col, hue='diabetes', bins=30, kde=True, ax=axes[i])
    axes[i].set_title(col)
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
# Remove empty subplots if any
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

X_num = df1.select_dtypes(include=['int64','float64'])  # numerical
X_cat = df1.select_dtypes(include=['object','category'])  # categorical
y = df1['diabetes']

# Feature Scaling
X_num_scaled = StandardScaler().fit_transform(X_num)
print(X_num_scaled)

# Categorical Encoding
from sklearn.preprocessing import OneHotEncoder
X_cat_encoded = OneHotEncoder(handle_unknown="ignore").fit_transform(X_cat).
    toarray()

import numpy as np
X_processed = np.hstack([X_num_scaled, X_cat_encoded])

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(X_processed, y, test_size=0.2,
     random_state=42, stratify=y)

# Model Training
```

```
model = Ridge(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Performance Metric
print("Accuracy :", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall   :", recall_score(y_test, y_pred))
print("F1 Score :", f1_score(y_test, y_pred))
```

Dataset 3 - Classification of Email spam and MNIST data

```
# Loading dataset
df = pd.read_csv("spambase_csv_Kaggle.csv")

# First five rows of the dataset
df.head()

# Dataset Columns details
df.info()

# Statistics summary of the numerical columns
df.describe()

# List of columns in the dataset
df.columns

# Number of rows and columns of the Dataset
df.shape

# Checking if any null / missing values present
df.isna().sum()

# Checking if any duplicate rows are found.
print("Duplicates in data: ", any(df.duplicated()))

# Remove duplicates rows.
df = df.drop_duplicates()
print("Duplicates in data: ", any(df.duplicated()))

# Checking the missing and null values...for train data
print("Any Missing Values: ", any(df.isnull().sum()))

# Separate features (X) and target (y)
X = df.drop('class', axis=1)
y = df['class']

# Boxplot for outliers.
# Select only 8 features
selected_features = X.columns[:8]
X_subset = X[selected_features]
n_cols = 4   # 4 plots per row
n_rows = 2   # 2 rows
fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 8))
axes = axes.flatten()
for i, col in enumerate(X_subset.columns):
    sns.boxplot(y=X_subset[col], ax=axes[i])
```

```
    axes[i].set_title(f'Boxplot of {col}')
    axes[i].set_xlabel('')
    axes[i].set_ylabel('Values')
# Remove empty plots if any
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

# Handling outliers by using capping.
cols = df.drop(columns='class', axis = 1)
for col in cols:
    upper_limit = df[col].quantile(0.95)
    lower_limit = df[col].quantile(0.05)
    df[col] = np.where(df[col]>upper_limit, upper_limit, np.where(df[col]<
   lower_limit, lower_limit, df[col]))

# Separate features (X) and target (y) again after handling outliers.
X = df.drop('class', axis=1)
y = df['class']

# Correlation analysis
corr = df.corr()

# Heatmap plot for correlation analysis
plt.figure(figsize=(12, 10))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm", square=True, annot_kws={
   "size": 8})
plt.title("Correlation Heatmap")
plt.show()

# Feature vs. target scatter plots
# Select numerical features (exclude target if numeric)
num_features = df.select_dtypes(include=['int64', 'float64']).columns
num_features = num_features.drop('class', errors='ignore')
# Select only first 8 features
selected_features = num_features[:8]
n_cols = 3, n_rows = 3
fig, axes = plt.subplots(n_rows, n_cols, figsize=(16, 15))
axes = axes.flatten()
for i, col in enumerate(selected_features):
    sns.scatterplot(x=df[col], y=df["class"], ax=axes[i], s=30, alpha=0.7)
    axes[i].set_title(f"{col} vs Target")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel("Class")
# Remove unused subplots (safe)
for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

# Checking the class balance
print("Class 1 has : ", df[df['class'] == 1].shape)
print("Class 0 has : ", df[df['class'] == 0].shape)

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2,
   random_state=42, stratify=y)
```

```
# Standardize numerical features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model Training
reg = LogisticRegression(max_iter=1000)
reg.fit(X_train_scaled, y_train)
y_pred = reg.predict(X_test_scaled)

# Logistic Regression Performance Metric
print("Logistic Regression Performance Metric")
print("Accuracy: ", accuracy_score(y_test, y_pred))
print("Recall: ", recall_score(y_test, y_pred))
print("Precision: ", precision_score(y_test, y_pred))
print("F1 Score: ", f1_score(y_test, y_pred))
```

Dataset 4 - Iris Dataset

```
# Correct column names (including Id temporarily)
column_names = ['id', 'sepal_length', 'sepal_width', 'petal_length', 'petal_width'
    , 'species']

# Read CSV:
# header=None -> because file structure is messy
# skiprows=[1] -> skip the second header row
df = pd.read_csv("iris.csv", header=None, names=column_names, skiprows=[1])

# Drop the Id column (not needed)
df.drop(columns=['id'], inplace=True)

# Convert numeric columns explicitly
num_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
df[num_cols] = df[num_cols].apply(pd.to_numeric, errors='coerce')

# 2. Inspect the data
print("First 5 rows (now correct):")
display(df.head())

print("\nData Info:")
print(df.info())

print("\nSummary Statistics:")
display(df.describe())
df = df.iloc[1:].reset_index(drop=True)
# Histogram \& BoxPlot
num_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']
ncols = 5
nrows = 2 * math.ceil(len(num_cols) / ncols)
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(20, 4*nrows))
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.histplot(df[col], kde=True, ax=axes[i])
    axes[i].set_title(f"{col} - Histogram")
    axes[i].set_xlabel(col)
    axes[i].set_ylabel('Count')
```

```
    sns.boxplot(x=df[col], ax=axes[i + ncols])
    axes[i + ncols].set_title(f"{col} - Boxplot")
    axes[i + ncols].set_xlabel(col)
    axes[i + ncols].set_ylabel('Count')
# Remove unused subplots
for j in range(len(num_cols), ncols):
    fig.delaxes(axes[j])
    fig.delaxes(axes[j + ncols])
plt.tight_layout()
plt.show()

# Confusion Matrix
num_cols = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width']

# Correlation matrix
corr = df[num_cols].corr()
plt.figure(figsize=(8,6))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Matrix - Iris Dataset")
plt.show()

# Class Distribution
fig, ax = plt.subplots(1, 1, figsize=(6,4))
# Count plot
sns.countplot(x='species', data=df, ax=ax)
ax.set_title("Class Distribution")
ax.set_xlabel('species')
ax.set_ylabel('count')
plt.tight_layout()
plt.show()

# BoxPlot
ncols = 5
nrows = math.ceil(len(num_cols) / ncols)
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(20, 4*nrows))
axes = axes.flatten()
for i, col in enumerate(num_cols):
    sns.boxplot(x='species', y=col, data=df, ax=axes[i])
    axes[i].set_title(f"{col} vs Species")
    axes[i].tick_params(axis='x', rotation=30)
# Remove unused subplots
for j in range(len(num_cols), len(axes)):
    fig.delaxes(axes[j])
plt.tight_layout()
plt.show()

# Feature - target split
X = df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']]
y = df['species']

# Encode target labels
le = LabelEncoder()
y_encoded = le.fit_transform(y)

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train - Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled,  y_encoded,
    test_size=0.2, random_state=42, stratify=y_encoded)

# Model Training
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Performance Metric
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=le.classes_))
```

Dataset 5 - Handwritten character recognition

```
# Load dataset
df = pd.read_csv("english.csv", names=['image_path','label'])

# shape of tha dataset
print(df.shape)
print(df['label'].value_counts())

# Count plot
sns.countplot(x='label', data=df)
plt.title("Class Distribution")
plt.show()

# Histogram
img_folder = r"C:\Users\SSN\Desktop\College_Sem\6th_Sem_Materials\ML Lab\
    Exercise_1\Img"
heights, widths = [], []
# Skip first row (invalid 'image') and process remaining
for path in df['image_path'][1:101]:  # first 100 valid images
    img_file = os.path.basename(path)  # get just the filename
    img_path = os.path.join(img_folder, img_file)
    if not os.path.exists(img_path):
        print("Image not found:", img_path)
        continue
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print("Failed to read:", img_path)
        continue
    h, w = img.shape
    heights.append(h)
    widths.append(w)
# Plot histograms
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(heights, bins=10, color='skyblue', edgecolor='black')
axes[0].set_title("Image Heights")
axes[0].set_xlabel("Pixels")
axes[0].set_ylabel("Count")
axes[1].hist(widths, bins=10, color='salmon', edgecolor='black')
axes[1].set_title("Image Widths")
axes[1].set_xlabel("Pixels")
axes[1].set_ylabel("Count")
plt.tight_layout()
plt.show()
```

```
print(df['image_path'][:10])
img_folder = r"C:\Users\SSN\Desktop\College_Sem\6th_Sem_Materials\ML Lab\
    Exercise_1\Img"

# Create subplots
fig, axes = plt.subplots(1, 5, figsize=(15, 4))
# Skip first invalid row and take first 5 valid images
for i, path in enumerate(df['image_path'][1:6]):
    # Get the filename only
    img_file = os.path.basename(path)
    img_path = os.path.join(img_folder, img_file)
    # Check if file exists
    if not os.path.exists(img_path):
        print("Image not found:", img_path)
        continue
    # Read image in grayscale
    img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print("Failed to read:", img_path)
        continue
    # Plot histogram
    axes[i].hist(img.ravel(), bins=50, color='gray')
    axes[i].set_title(f"Image {i+1}")
    axes[i].set_xlabel("Pixel Value")
    axes[i].set_ylabel("Count")
plt.tight_layout()
plt.show()
img_folder = r"C:\Users\SSN\Desktop\College_Sem\6th_Sem_Materials\ML Lab\
    Exercise_1\Img"
# get unique class labels
classes = df['label'].unique()
fig, axes = plt.subplots(1, len(classes), figsize=(5*len(classes), 4))
for i, label in enumerate(classes):
    imgs = []
    # skip first row (invalid) and take first 20 images
    for path in df[df['label']==label]['image_path'][1:21]:
        img_file = os.path.basename(path)
        img_path = os.path.join(img_folder, img_file)
        if not os.path.exists(img_path):
            print("Image not found:", img_path)
            continue
        img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            print("Failed to read:", img_path)
            continue
        imgs.append(img)
    if len(imgs) == 0:
        print(f"No valid images found for class {label}")
        continue
    mean_img = np.mean(imgs, axis=0)
    axes[i].imshow(mean_img, cmap='gray')
    axes[i].set_title(f"Mean Class {label}")
    axes[i].axis('off')
plt.tight_layout()
plt.show()
exists = []
for path in df['image_path']:
    exists.append(os.path.exists(path))
```

```
fig, ax = plt.subplots(1,1, figsize=(5,4))
sns.countplot(x=exists, ax=ax)
ax.set_xticklabels(['Missing','Exists'])
ax.set_title("Image Availability")
plt.tight_layout()
plt.show()
```

3 Output Screenshots

Dataset 1 - Loan amount prediction

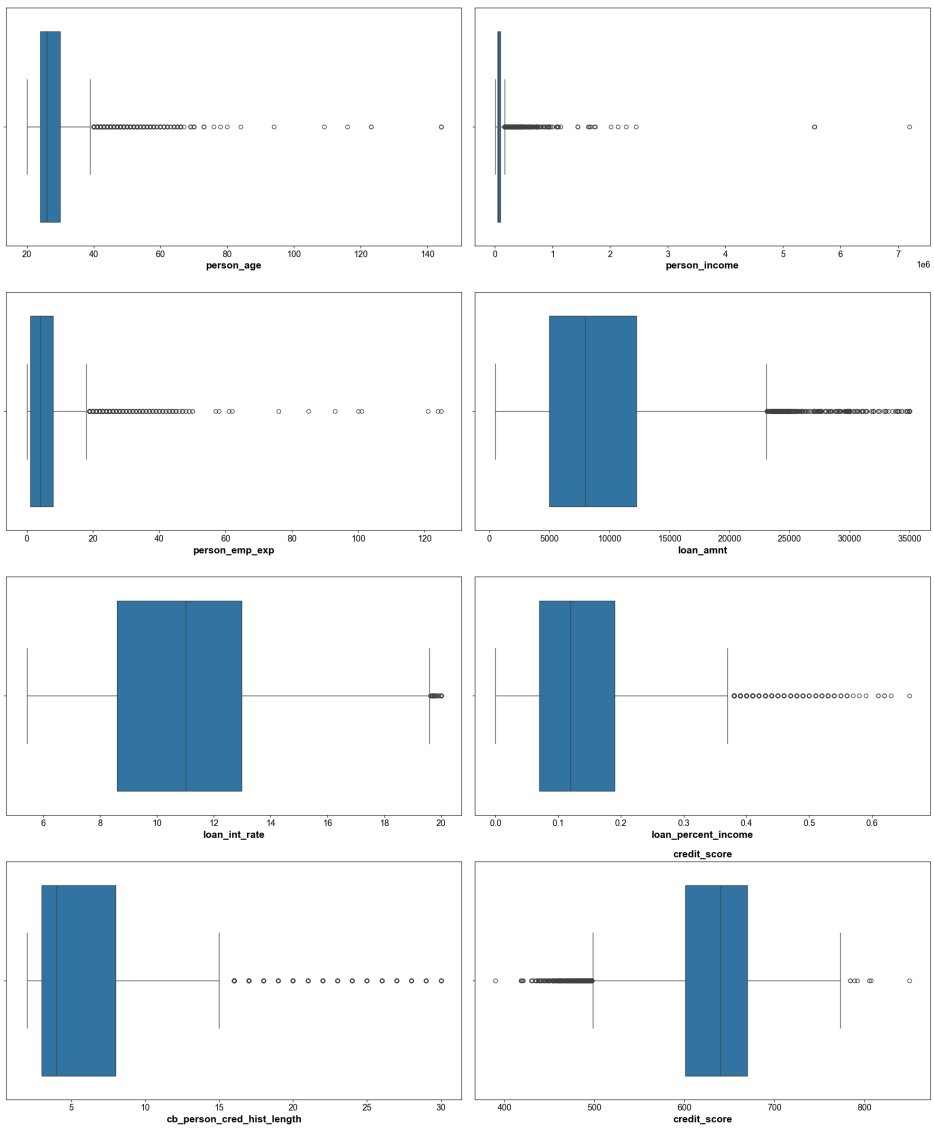

Figure 1: Boxplot for numerical columns

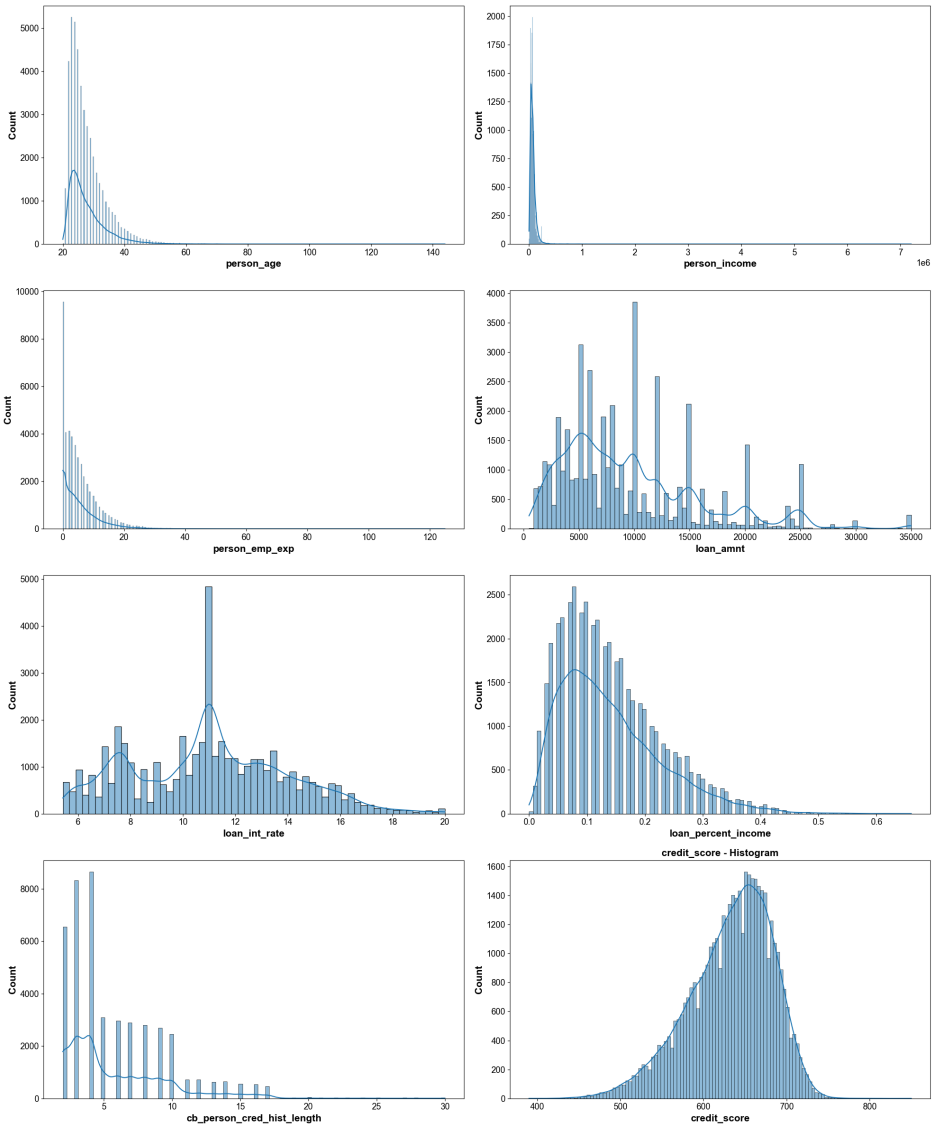

Figure 2: Histogram with kde for numerical columns

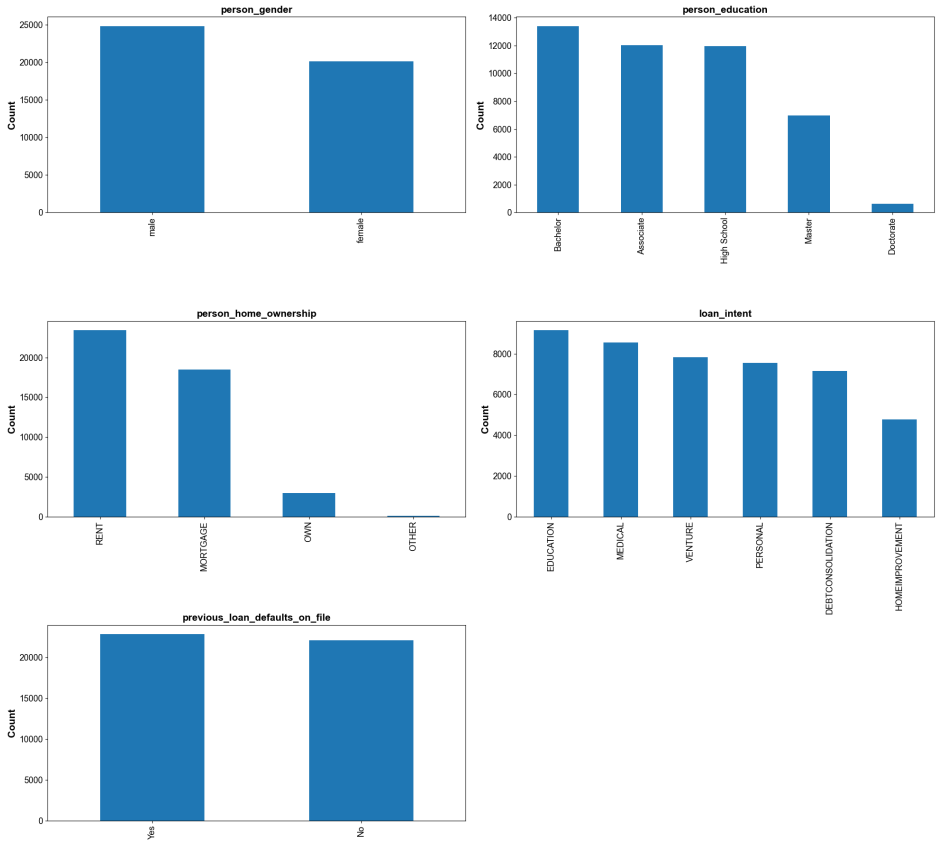

Figure 3: Bar plot for categorical columns

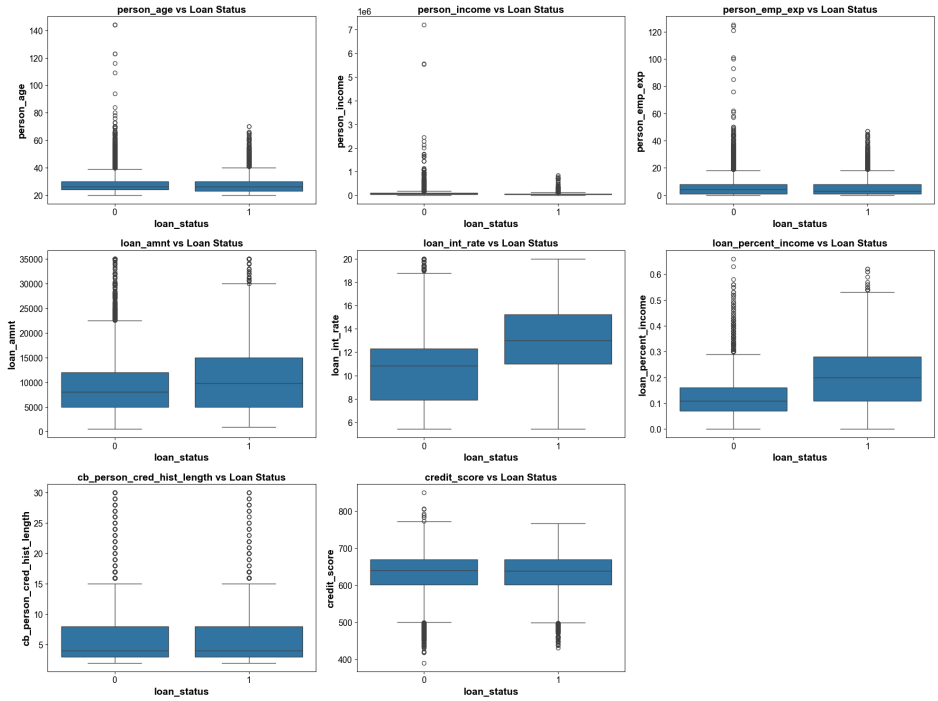

Figure 4: Box Plot with columns vs loan status

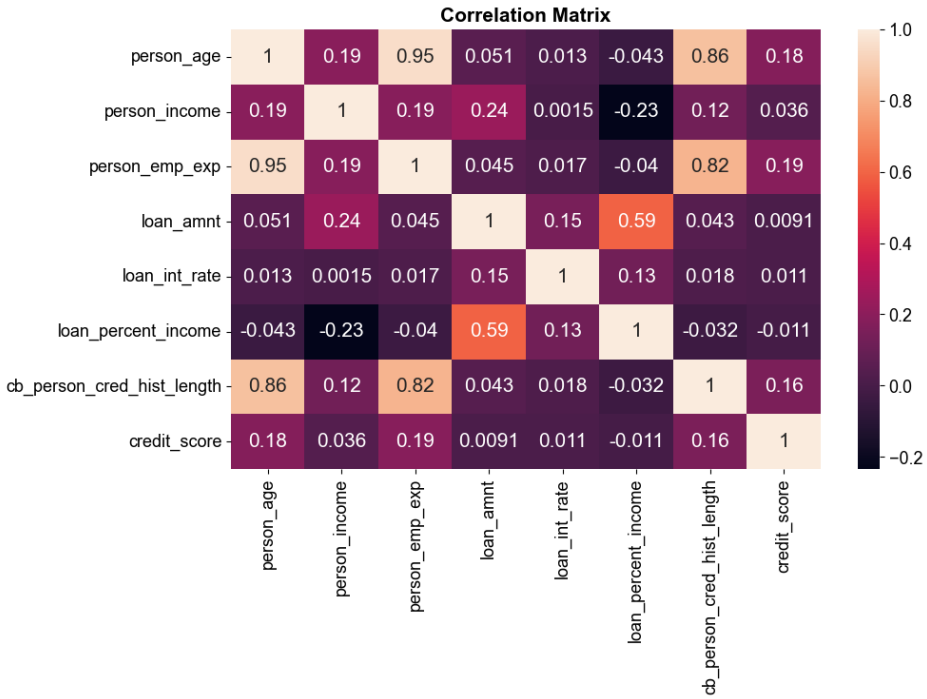

Figure 5: Correlation Matrix

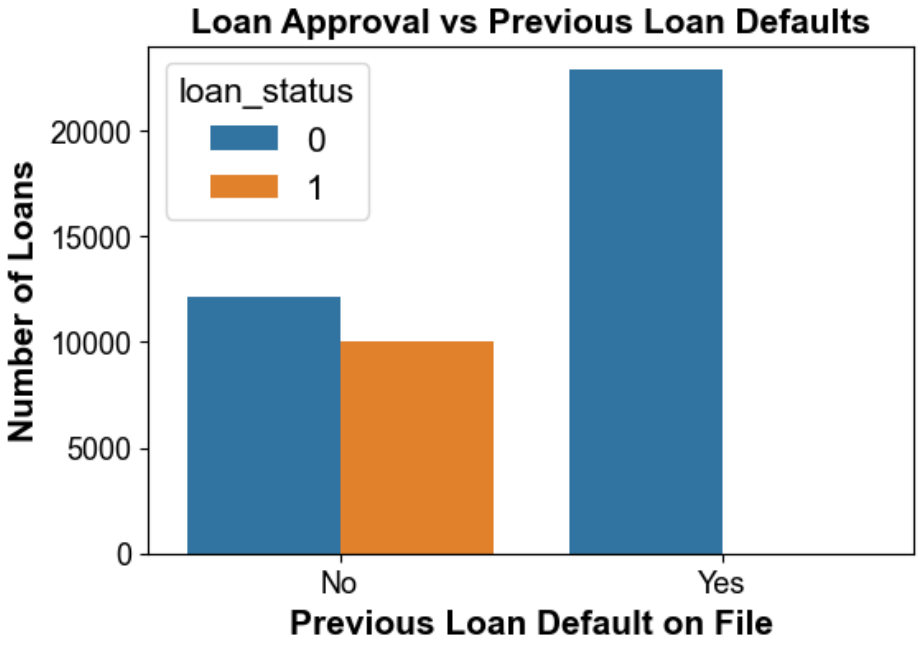

Figure 6: Loan Approval vs Previous Loan Defaults

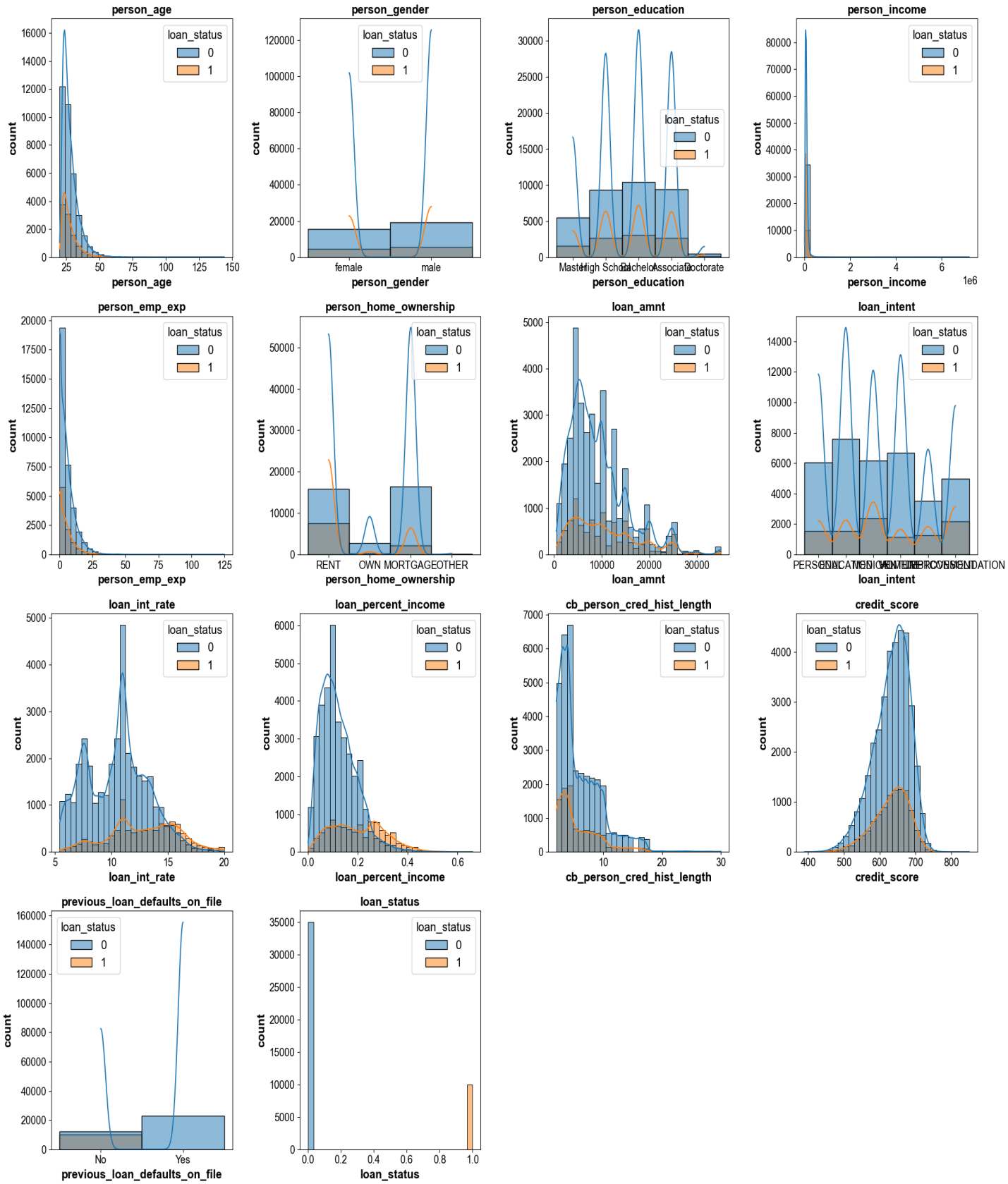

Figure 7: Histogram plots for all columns

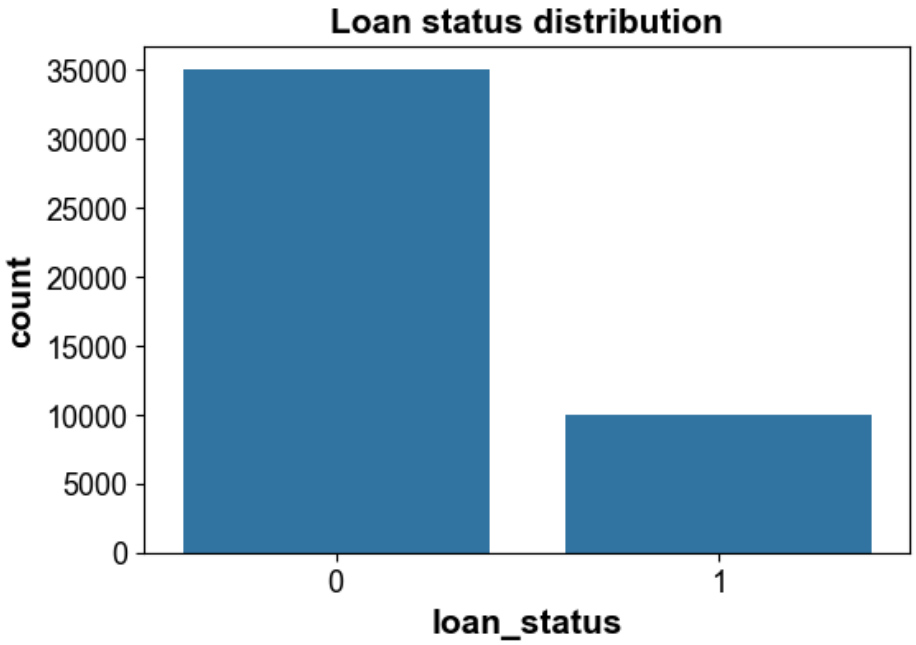

Figure 8: Loan Status Distribution

Dataset 2 - Predicting Diabetes

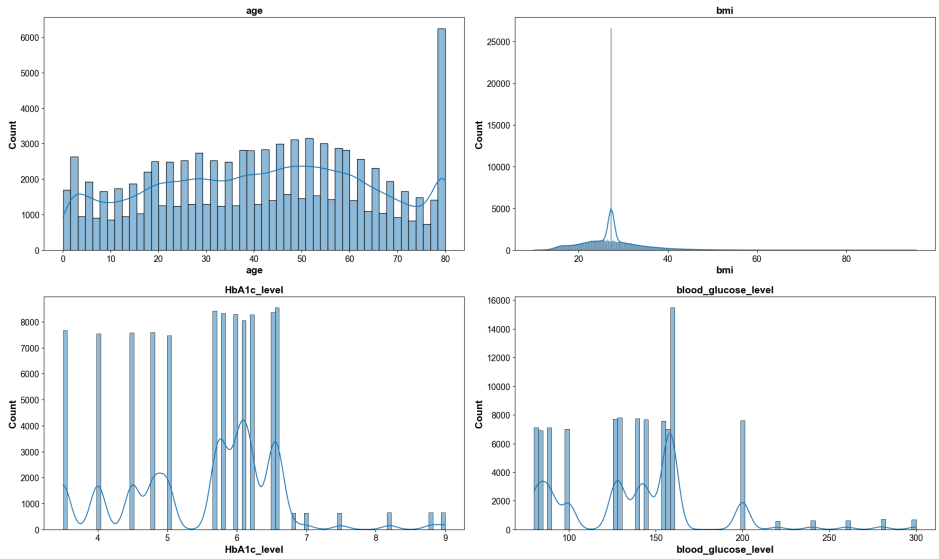

Figure 9: Histogram plots for numerical features.

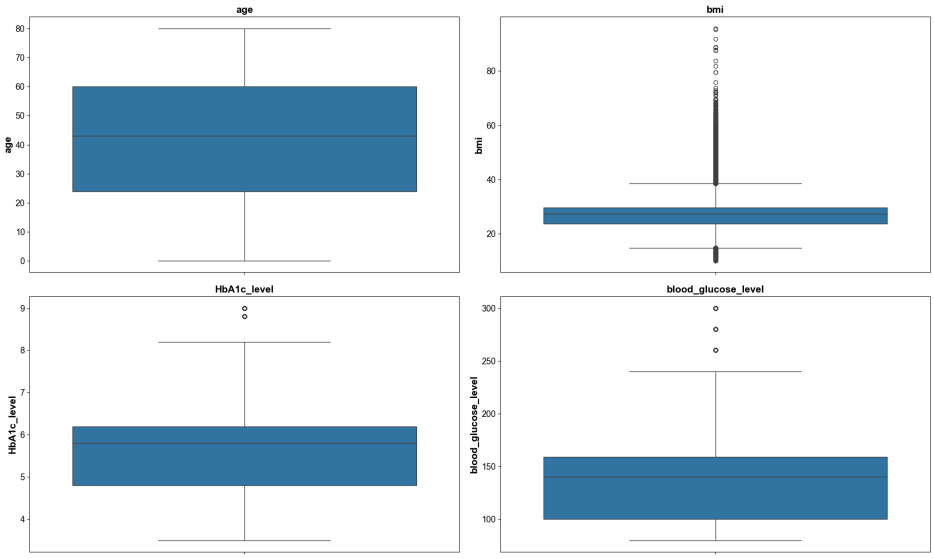

Figure 10: Box Plot

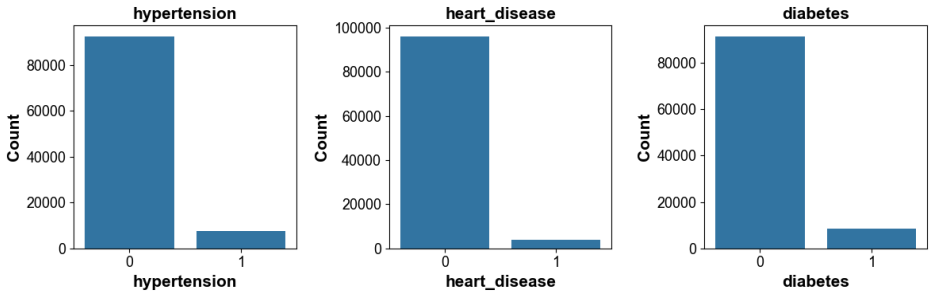

Figure 11: Bar Plots

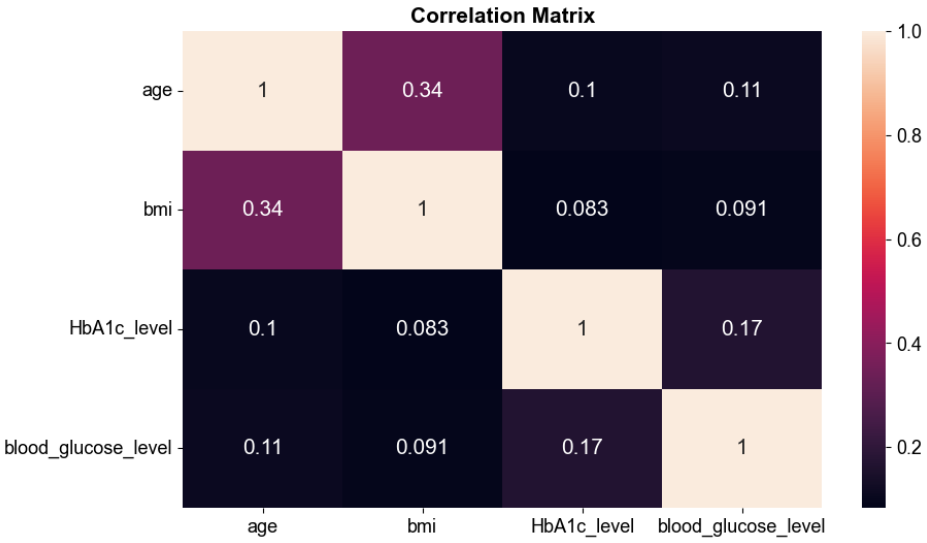

Figure 12: Correlation matrix

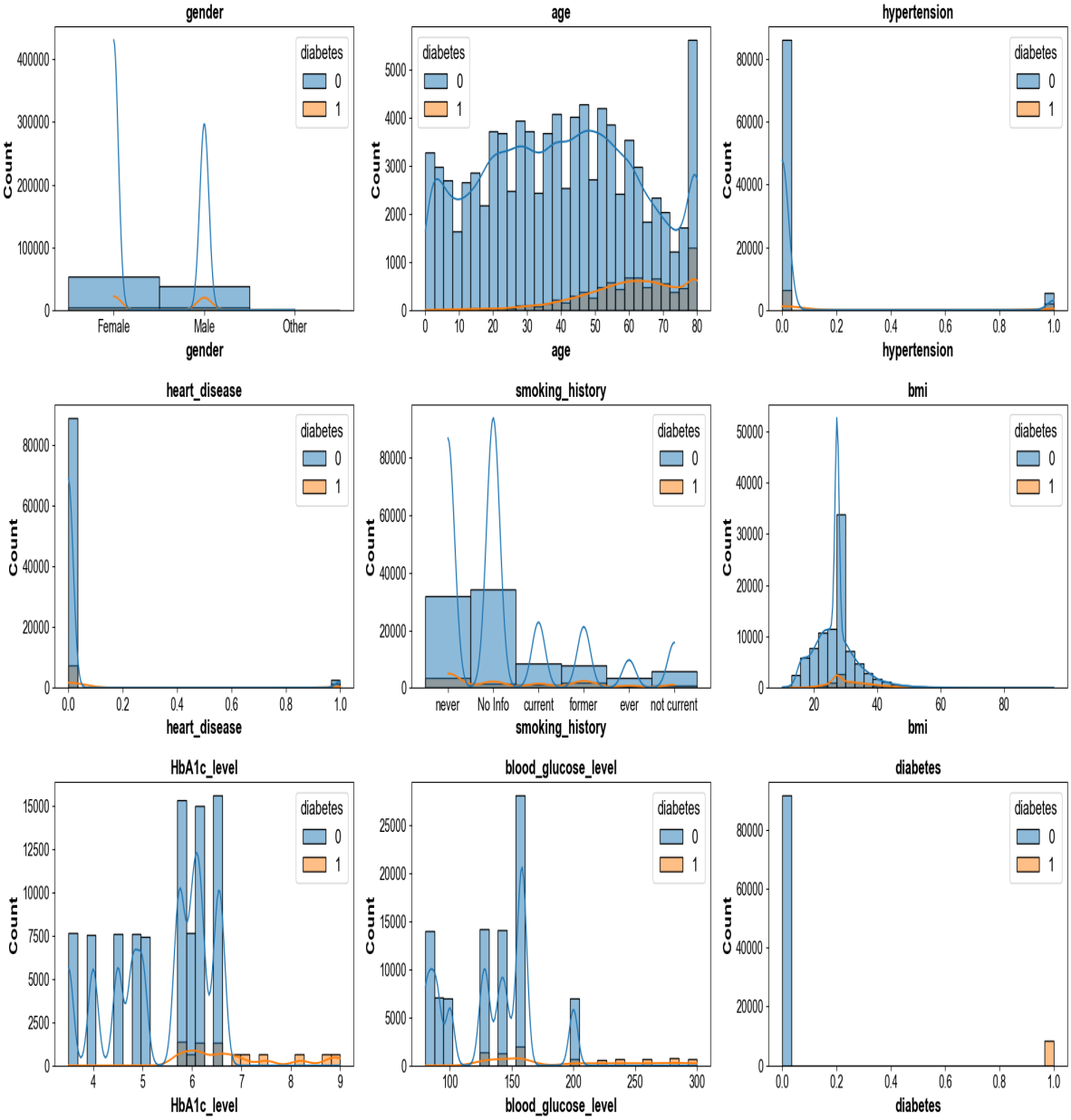

Figure 13: Histogram plots

Dataset 3 - Classification of Email spam and MNIST data

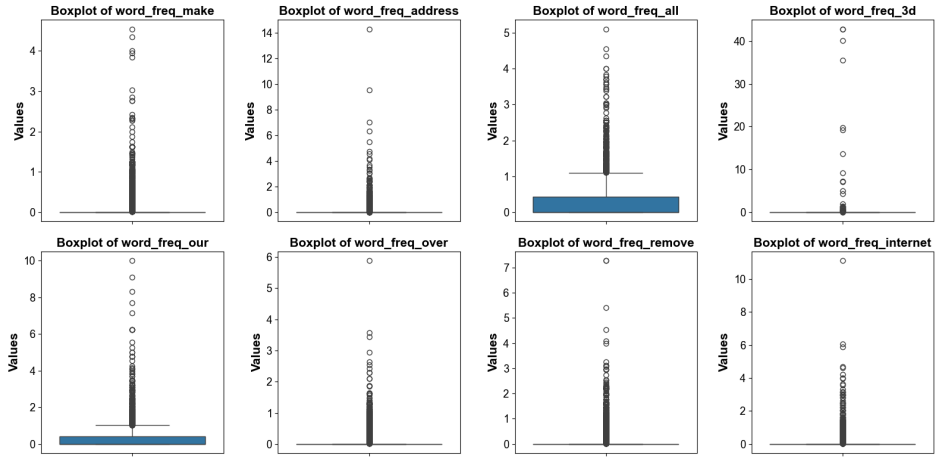

Figure 14: Box plot

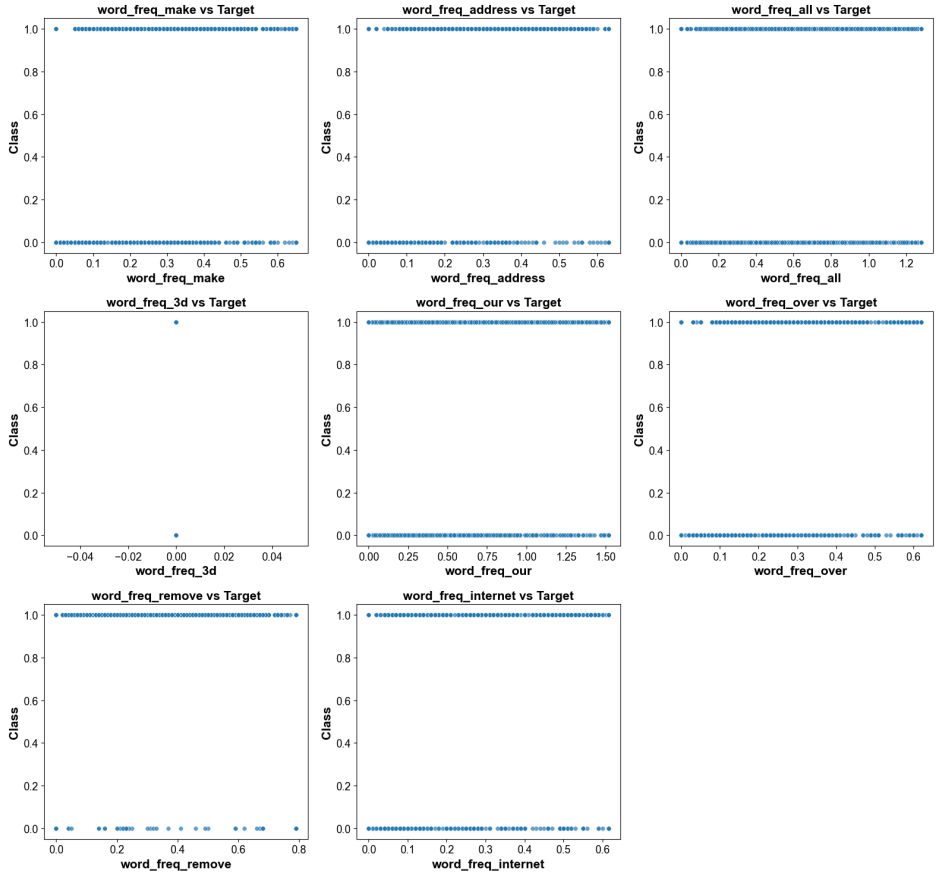

Figure 15: Feature vs. target scatter plots

Dataset 4 - Iris Dataset

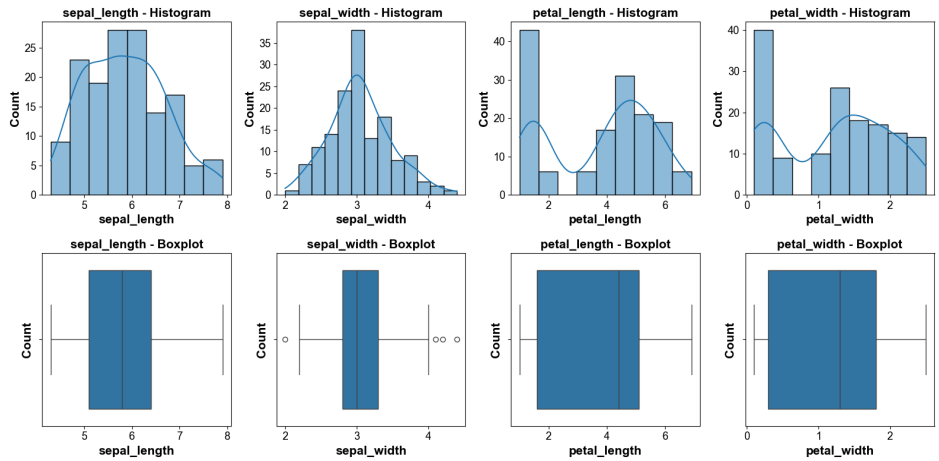

Figure 16: Histogram and Boxplot

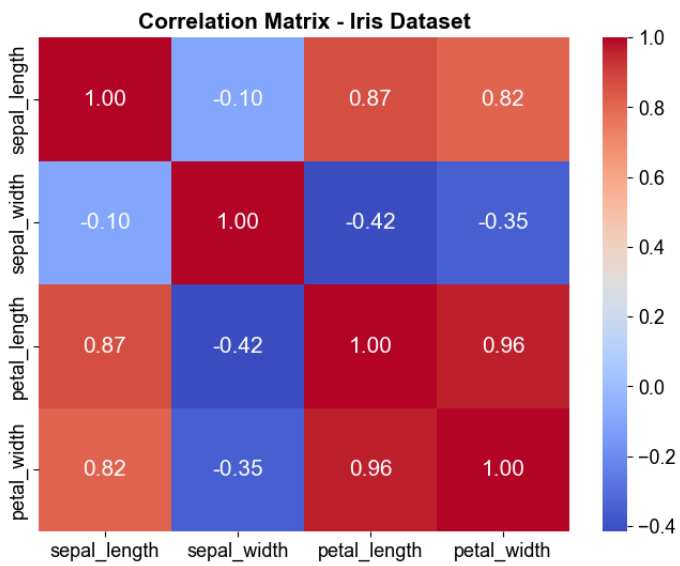

Figure 17: Correlation Matrix - Iris Dataset

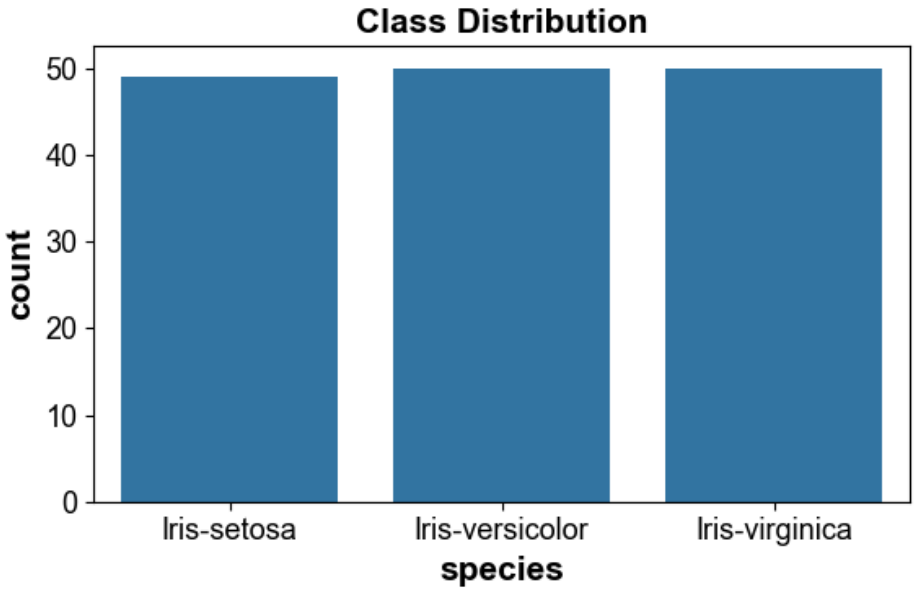

Figure 18: Class Distribution of species

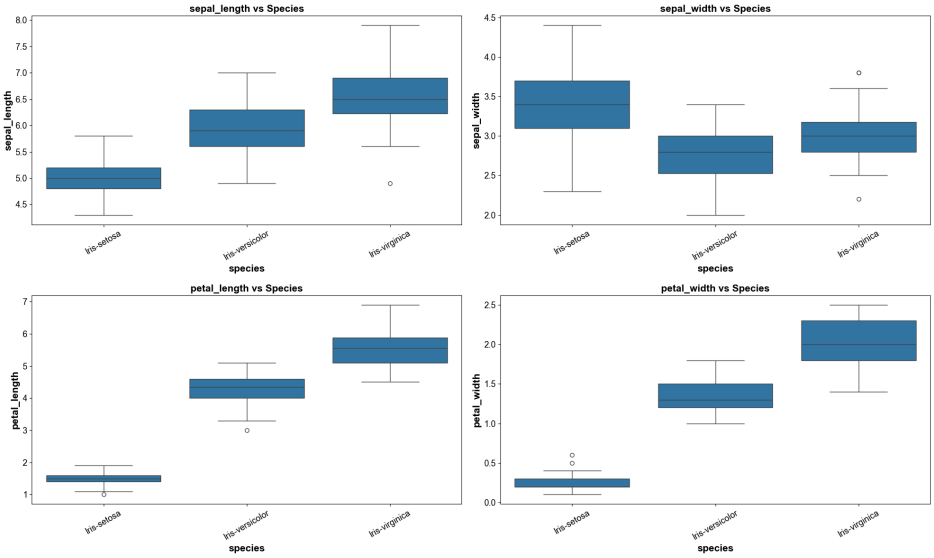

Figure 19: Box plot for features vs target feature

Dataset 5 - Handwritten character recognition

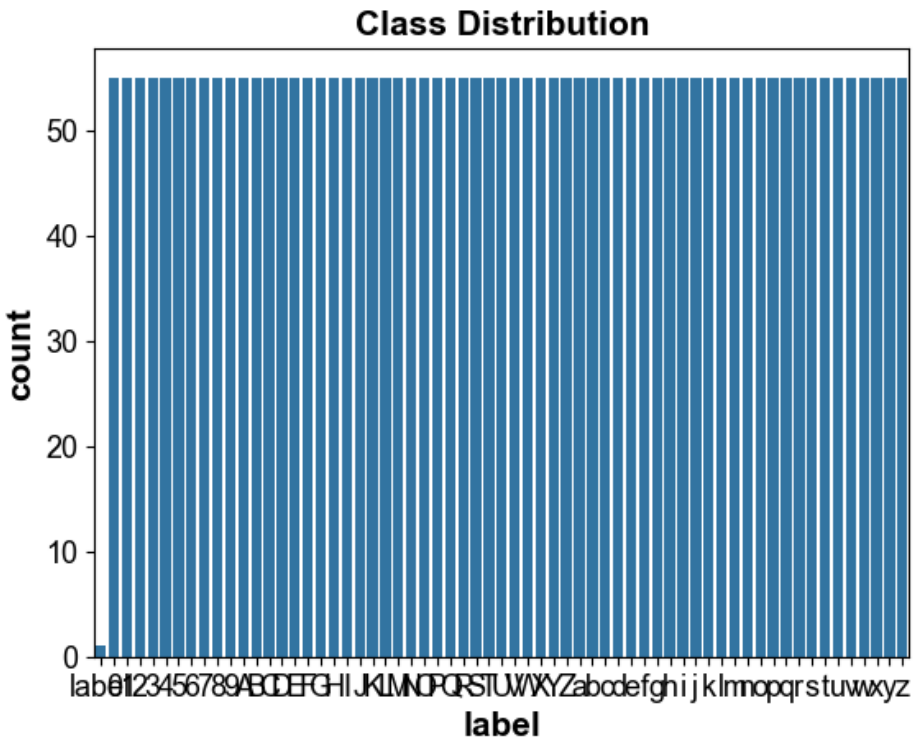

Figure 20: Class Distribution

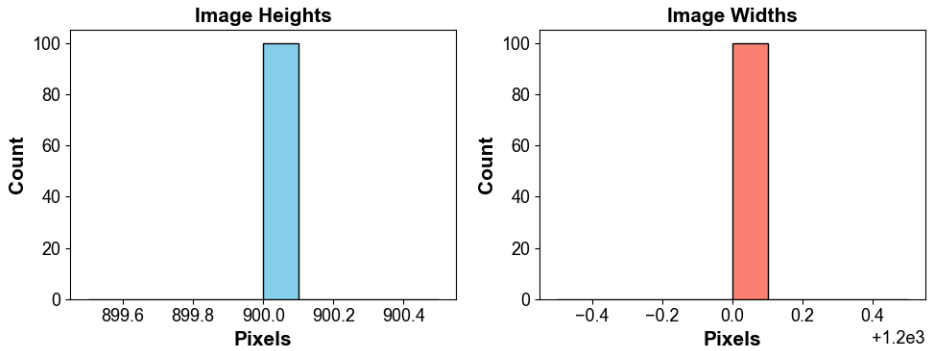

Figure 21: Histogram plot of Image Heights and Widths

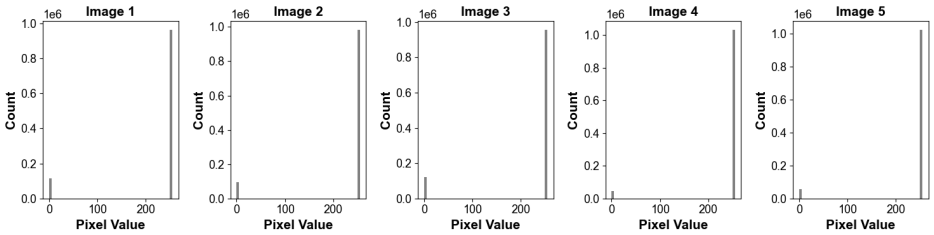

Figure 22: Histogram for pixel value of different images.

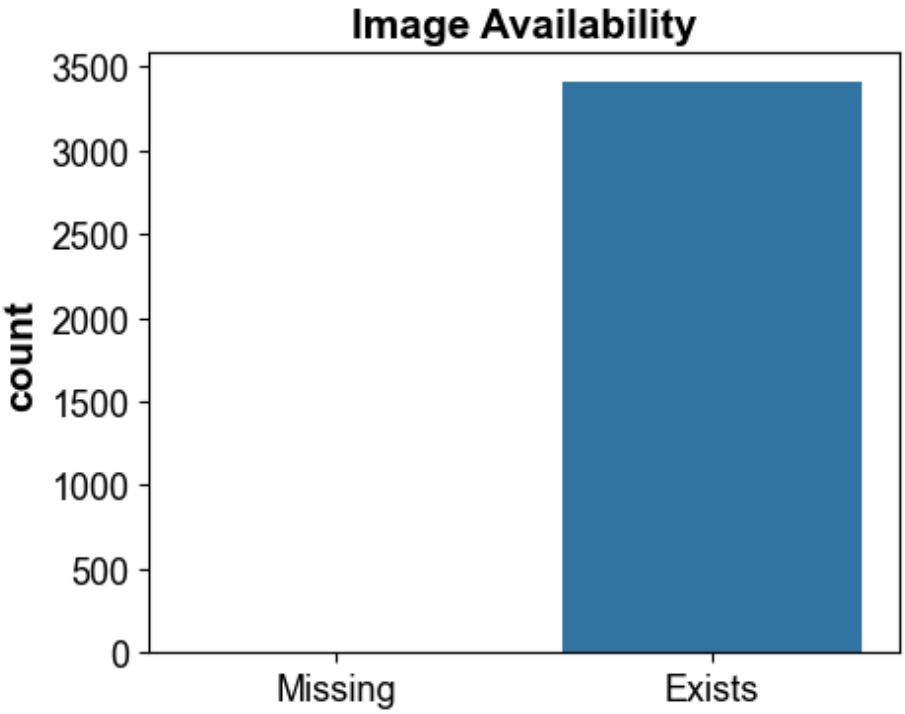

Figure 23: Image Availability

4 Inference Table

Table 1: Task Performed & its details.

Dataset	Type of ML Task	Feature Selection Technique	Suitable ML Algorithm
Iris Dataset	Classification	SelectKBest (ANOVA F-Test)	Logistic Regression
Loan Amount Prediction	Regression	Correlation-based	Linear Regression
Predicting Diabetes	Regression	Lasso Regression	Ridge Regression
Classification of Email Spam	Classification	Chi-square	Naive Bayes
Handwritten Character Recognition / MNIST	Classification	LDA	Random Forest, CNN

5 Learning Outcome.

- Understand and apply essential Python libraries such as **NumPy, Pandas, SciPy, Scikit-learn, and Matplotlib** for data handling, numerical computation, and visualization.
- Perform **data preprocessing** tasks including handling missing values, feature encoding, normalization, and standardization as part of the machine learning workflow.
- Explore and analyze real-world datasets using **Exploratory Data Analysis (EDA)** techniques such as summary statistics, histograms, scatter plots, box plots, and heatmaps.
- Identify and differentiate between various **machine learning tasks** such as classification, regression, and prediction based on the nature of the dataset.
- Select appropriate **machine learning models** for different problem statements, including regression and classification problems.
- Apply **feature selection techniques** to identify relevant attributes that improve model performance.
- Split datasets into training and testing sets and evaluate model performance using suitable **performance metrics**.
- Gain practical exposure to the complete **end-to-end machine learning pipeline**, from data acquisition to model evaluation and interpretation of results.